

Table Of Content

1. INTRODUCTION.....	2
1.1. Clinical Background and Anatomy of Bell's Palsy.....	2
1.2. The Core Problem Statement.....	3
1.3. Primary Project Objectives.....	3
1.4. Scope and Limitations.....	3
2. LITERATURE REVIEW AND SYSTEM ANALYSIS	4
2.1. Review of Traditional Clinical Grading Systems.....	4
2.2. Existing System vs. Proposed AI System Analysis.....	4
2.3. Comprehensive Feasibility Study.....	4
3. THEORETICAL FRAMEWORK AND TECHNOLOGY STACK.....	5
3.1. Python and OpenCV (Computer Vision).....	5
3.2. Deep Learning and Google MediaPipe Face Mesh.....	5
3.3. NumPy for Array Processing and Noise Reduction.....	5
4. SYSTEM DESIGN AND ARCHITECTURE.....	6
4.1. The 6-Stage Algorithmic Data Pipeline.....	6
4.2. Landmark Clustering Strategy (Sensor Noise Mitigation).....	7
5. IMPLEMENTATION DETAILS AND MATHEMATICAL MODELING.....	8
5.1. Working Principle: The Relative Displacement Delta.....	8
5.2. The Core Mathematical Equations.....	8
5.3. The Finite State Machine (FSM) Logic.....	10
6. USER INTERFACE AND AR INTEGRATION.....	10
7. Current Working.....	11
7.1. Phase 1: Video capturing and score generation.....	11
7.3. Phase 2: Data Storing and Visualization.....	12
7.2. Limitations.....	13
8. CONCLUSION AND FUTURE SCOPE.....	13
8.1. Conclusion.....	13
8.2. Future Scope.....	14
9. REFERENCES.....	14

Github Link: <https://github.com/shashankp89/Bell-s-Palsy-Rehabilitation-Assistant/tree/main>

ABSTRACT

Bell's Palsy is an acute, idiopathic neurological condition characterized by the sudden onset of unilateral facial paralysis or weakness. The condition fundamentally disrupts a patient's physical and psychological well-being, severely impacting their ability to communicate, eat, and express emotion. Effective rehabilitation mandates a strict regimen of targeted physical therapy exercises to stimulate the facial nerve and retrain the affected musculature. However, the standard clinical approach to at-home recovery relies entirely on qualitative visual self-assessment—patients performing exercises in front of a mirror and estimating their progress. This subjective methodology lacks precise, quantitative feedback, frequently resulting in a loss of patient motivation and poor adherence to therapy protocols.

This comprehensive progress report details the conceptualization, system architecture, and development of an AI-based rehabilitation assistant engineered to resolve this clinical deficiency. Utilizing a Python-based processing pipeline, the OpenCV library, and Google MediaPipe's Convolutional Neural Network (CNN), the proposed system maps 468 high-fidelity topographic landmarks on the human face in real-time using standard consumer webcam hardware. By grouping these nodes into stable clusters and applying geometric Euclidean distance formulas across five standardized clinical exercises, the system computes a highly accurate, numerical "Symmetry Score."

The backend processing engine, representing Phase 1 of the project, is currently fully operational. It successfully tracks relative muscle displacement, filters optical sensor noise, and provides immediate Augmented Reality (AR) feedback to the user. While the system currently processes and outputs data in real-time, Phase 2 (Future Scope) will finalize the data serialization process—structuring the diagnostic scores into JSON formatting—and implement a graphical web dashboard using React.js for long-term clinical review.

1. INTRODUCTION

1.1 Clinical Background and Anatomy of Bell's Palsy

To fully grasp the engineering requirements of this software, it is vital to understand the anatomical failure it attempts to monitor. The human face is innervated by Cranial Nerve VII (the facial nerve). This complex nerve travels through a narrow bony canal in the skull beneath the ear. In cases of Bell's Palsy, this nerve becomes inflamed, swollen, or compressed—often due to a viral infection.

When the nerve is compressed, the electrical signals traveling from the brain to the facial muscles are blocked. This results in unilateral (one-sided) flaccid paralysis. The patient loses the ability to contract the Frontalis muscle (raising the eyebrows), the Orbicularis Oculi (closing the eye), and the Zygomaticus major (smiling). Recovery involves neuroplasticity—the brain and nerve slowly healing and re-establishing these electrical pathways over weeks or months.

1.2 The Core Problem Statement

The fundamental issue in Bell's Palsy recovery is the lack of visible, day-to-day progress. Because nerve healing is microscopic, a patient might only regain a fraction of a millimeter of muscle mobility in a 48-hour period.

Currently, a patient goes home, stands in front of a mirror, and attempts to smile. The human visual cortex is not equipped to detect a 1 or 2-millimeter spatial improvement in a moving target. Because the patient cannot visually confirm their progress, psychological fatigue sets in. They assume the physical therapy is failing, leading to depression and a failure to complete their prescribed exercises. The medical community lacks an accessible, at-home digital "measuring tape" capable of detecting these micro-improvements to validate the patient's hard work.

1.3 Primary Project Objectives

This project aims to bridge the gap between clinical physical therapy and modern computer science by achieving the following objectives:

1. **Develop a Real-Time Vision Engine:** Engineer a robust Python software script capable of interpreting standard webcam feeds and accurately mapping the spatial geometry of the human face with low latency.
2. **Translate Kinematics into Mathematics:** Develop and implement algorithms that measure the relative physical distance a muscle contracts or extends during an exercise, converting that spatial data into a standardized percentage score.
3. **Design an Interactive User Loop:** Create a State Machine that guides the user through a structured medical routine, ensuring exercises are performed and measured consistently.
4. **Establish Data Architecture:** Structure the output of these mathematical calculations so they can be seamlessly serialized and exported into a digital database for long-term clinical review.

1.4 Scope and Limitations

The current scope encompasses the design, implementation, and synthetic validation of the core Python backend vision engine. The system is programmed to evaluate five primary muscular targets based on standard physical therapy protocols. It is vital to note that this software is strictly a kinematic progress tracker; it is not designed to diagnose Bell's Palsy, nor can it replace the professional medical advice of a neurologist. Furthermore, creating a persistent cloud database and a graphical web dashboard are scoped as future aspects for Phase 2 of the development lifecycle.

2. LITERATURE REVIEW AND SYSTEM ANALYSIS

2.1 Review of Traditional Clinical Grading Systems

In clinical settings, doctors evaluate facial paralysis using subjective scales, the most common being the House-Brackmann Facial Nerve Grading System. This scale grades patients from I (Normal) to VI (Total Paralysis). Another common tool is the Sunnybrook Facial Grading System, which relies on a clinician assigning point values to different resting and moving facial features.

While these scales are standard, they suffer from high inter-observer variability. Two different doctors might grade the same patient differently. Furthermore, these scales require a clinic visit and cannot be used by the patient on a daily basis at home.

2.2 Existing System vs. Proposed AI System Analysis

To clearly justify the engineering and computational effort required to build this software, Table 1 outlines the severe limitations of traditional physical therapy compared to the proposed automated AI pipeline.

Table 1: Comprehensive Comparative System Analysis

Feature/Parameter	Existing System (Traditional Mirror Method)	Proposed System (AI-Driven Software Engine)
Measurement Method	Qualitative visual estimation by the untrained patient.	Quantitative, deterministic mathematical coordinate tracking.
Data Logging	Manual paper diaries, which are often lost, forgotten, or fabricated.	Automated data structuring ready for JSON serialization.
Feedback Latency	Delayed significantly; the patient must wait weeks for a neurologist's opinion.	Immediate; real-time percentage scores displayed directly on the screen.
Micro-Trend Detection	Exceptionally poor; unable to spot millimeter-level changes.	High; tracks exact relative pixel displacement to establish recovery curves over time.

2.3 Comprehensive Feasibility Study

Before writing the core engine, a feasibility study was conducted to ensure the project could be executed within the timeframe of a 4th-semester curriculum.

- **Technical Feasibility:** The project relies on Python, a highly versatile and well-documented language. By utilizing pre-trained, open-source Machine Learning models (Google MediaPipe) rather than training a Convolutional Neural Network from scratch, the software avoids the need for massive datasets and expensive GPU clusters.
- **Economic Feasibility:** The system is entirely cost-effective. It requires no specialized medical hardware, infrared sensors, or depth cameras. It is designed to run efficiently

on standard consumer-grade laptops using built-in 720p or 1080p webcams, making it highly accessible to the average patient.

- **Operational Feasibility:** The keyboard-driven interface ensures that even patients with limited technical literacy can operate the calibration and recording phases smoothly.

3. THEORETICAL FRAMEWORK AND TECHNOLOGY STACK

The backend of this system is developed using a robust, industry-standard stack of computer vision and mathematical libraries. Understanding the theory behind these tools is essential to understanding the system's architecture.

3.1 Python and OpenCV (Computer Vision)

Python 3.x serves as the foundational programming language due to its unparalleled support for machine learning integrations. OpenCV (Open Source Computer Vision Library) is utilized for all hardware interfacing. OpenCV acts as the "eyes" of the system. It is responsible for opening the webcam, capturing the video stream frame-by-frame as a multidimensional matrix of BGR (Blue, Green, Red) pixel values, and drawing the Augmented Reality (AR) text directly onto the video feed.

3.2 Deep Learning and Google MediaPipe Face Mesh

The core intelligence of the system relies on Google MediaPipe. MediaPipe Face Mesh is a highly optimized Machine Learning model that estimates 468 3D facial landmarks in real-time.

Under the hood, this utilizes a Convolutional Neural Network (CNN). A CNN works by passing the webcam image through multiple "filters" that learn to recognize edges, shadows, and shapes. MediaPipe's specific architecture is lightweight enough to run on a standard laptop CPU, predicting the exact X, Y, and Z spatial coordinates of the lips, eyes, and eyebrows in milliseconds.

3.3 NumPy for Array Processing and Noise Reduction

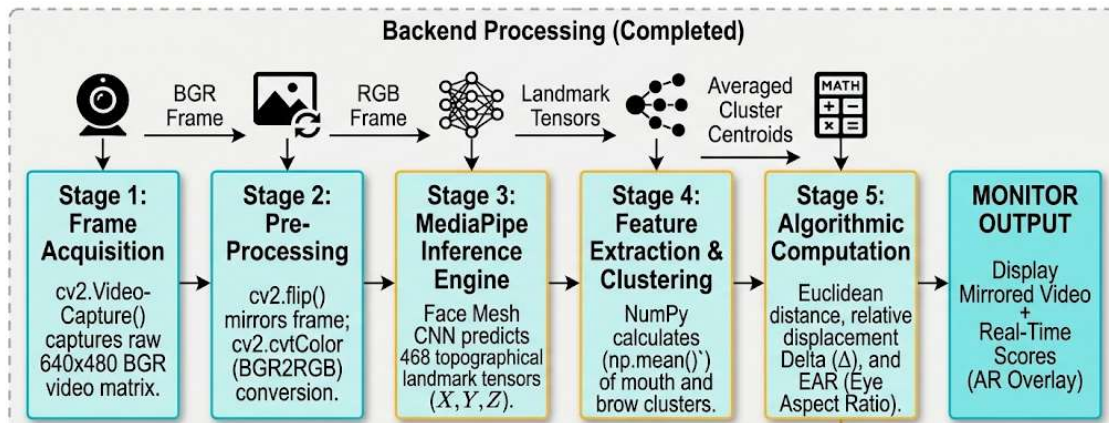
NumPy is a foundational Python library used for high-performance multidimensional array processing. In this project, NumPy is critical for data smoothing. Webcams inherently have sensor noise, which causes pixels to "jitter." If the software tracked a single pixel, this jitter would ruin the medical data. NumPy is used to instantly calculate the mathematical averages of large sets of coordinates, providing a stable, jitter-free center point for the algorithm to track.

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 The 6-Stage Algorithmic Data Pipeline

The software architecture is strictly modular, designed as a linear data pipeline. Data enters as raw light hitting a camera sensor, gets mathematically processed, and exits as a clinical score.

1. **Stage 1: Frame Acquisition:** The OpenCV VideoCapture function continuously polls the hardware webcam, pulling raw 640x480 resolution video matrices into the system's memory buffer.
2. **Stage 2: Pre-Processing:** Raw video frames are unprepared for AI analysis. The script first mirrors the frame horizontally using `cv2.flip` so it acts like a real mirror for the user. It then converts the color space from OpenCV's default BGR to RGB, which is the strict requirement for the MediaPipe CNN.
3. **Stage 3: Inference Engine (The AI):** The normalized RGB matrix is passed to the MediaPipe Face Mesh model. The neural network scans the image topography and generates a list of 468 spatial landmark nodes.
4. **Stage 4: Feature Extraction:** The Python script parses this massive list of nodes and extracts only the specific X and Y coordinate data necessary for the prescribed clinical exercises.
5. **Stage 5: Algorithmic Computation:** An internal Finite State Machine (FSM) dictates the flow. When the user signals they are performing an exercise, the script applies geometric distance formulas to the extracted coordinates to measure the physical stretch of the muscles.
6. **Stage 6: Data Serialization (FUTURE ASPECT):** The final calculated scores will be compiled, formatted into a key-value pair structure, and exported to a structured data.json file for permanent database storage.



4.2 Landmark Clustering Strategy (Sensor Noise Mitigation)

As mentioned, standard webcams suffer from pixel noise and dynamic lighting shifts. If the system only tracked a single landmark node on the corner of the mouth, a passing shadow might trick the AI into shifting that node by 3 pixels. In a medical context, a 3-pixel error could completely invalidate the Symmetry Score.

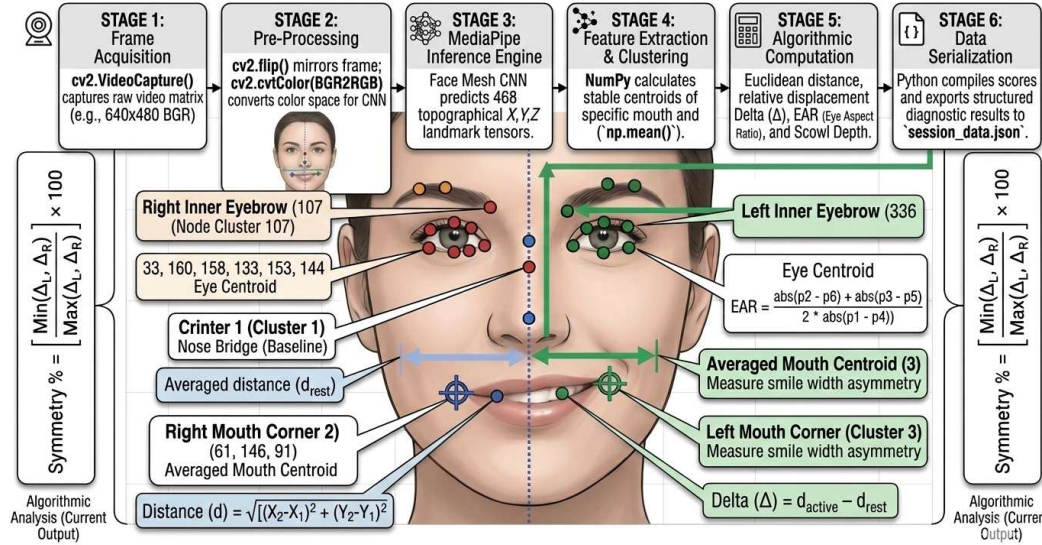
To solve this, the system employs **Centroid Clustering**. Instead of tracking one node, the software groups multiple adjacent nodes into a single "Cluster" and uses NumPy to calculate the arithmetic mean of their X and Y coordinates. If one node in the cluster glitches due to a shadow, the overall average center (the centroid) remains highly stable.

Table 2: MediaPipe Landmark Clustering Assignments

Target Facial Feature	MediaPipe Node IDs (The Clusters)	Engineering Purpose
Vertical Midline	Nodes 168, 1, 4	Establishes the static center vertical axis down the bridge of the nose.
Mouth Corners	Right: 61, 146, 91 / Left: 291, 375, 321	Tracks horizontal Zygomaticus extension during smiling and puckering.
Eyebrow Arches	Right: 105, 107, 70 / Left: 334, 336, 300	Tracks vertical Frontalis elevation during brow raises.
Eyelids	Right: 159, 145 / Left: 386, 374	Calculates the Eye Aspect Ratio (EAR) for blinking and squeezing.

5. IMPLEMENTATION DETAILS AND MATHEMATICAL MODELING

This chapter details the exact geometric logic the software utilizes to determine the extent of facial paralysis.



5.1 Working Principle: The Relative Displacement Delta

Because webcams sit at different, unpredictable distances from the user's face, the system cannot measure absolute real-world units like millimeters. If the user leans closer to the camera, their face appears larger, which would ruin absolute measurements.

Instead, the system relies on **Relative Pixel Displacement**.

The process is two-fold:

1. **Calibration:** The user relaxes their face. The system records the relative pixel distance from the Static Midline to the facial clusters. This establishes the patient's unique "Zero State" or Baseline.
2. **Active Measurement:** The user performs an exercise (e.g., a wide smile). The system records the new, maximum extended distance.

By subtracting the Baseline position from the Active position, we find the **Delta (Δ)**. The Delta represents the total relative distance the muscle successfully contracted.

5.2 The Core Mathematical Equations

The entire analytical capability of the engine relies on three primary geometric equations applied to the pixel coordinates.

Equation 1: Spatial Displacement (Euclidean Distance)

To find the direct linear distance between two points (such as the center of the nose and the corner of the mouth), the system uses the standard Euclidean distance formula across a 2D Cartesian plane:

$$\text{Distance (d)} = \sqrt{[(X2 - X1)^2 + (Y2 - Y1)^2]}$$

By calculating this distance during the resting phase and the active phase, the script can isolate the exact pixel shift caused by the muscle firing.

Equation 2: Eye Aspect Ratio (EAR)

Bell's Palsy patients frequently lose the ability to fully close their affected eye. This is a severe clinical issue, as an inability to blink leads to corneal drying and potential blindness. Measuring eye closure using standard distance is difficult because the user's head might tilt.

To measure eye closure safely, the system utilizes the Eye Aspect Ratio (EAR). This formula divides the vertical height of the eye by the horizontal width of the eye.

$$EAR = (|P2 - P6| + |P3 - P5|) / (2 \times |P1 - P4|)$$

(Where P1 through P6 represent the coordinates of the specific eyelid landmarks).

When the eye is open, the EAR is a stable higher number (e.g., 0.3). As the eye closes, the vertical height approaches zero, driving the entire EAR calculation toward zero. If the software detects that the EAR on the paralyzed side fails to drop below a strict threshold (e.g., 0.1), it logs a failure to close the eye.

Equation 3: The Hemispheric Symmetry Percentage Score

This is the final, ultimate output shown to the patient. It is designed to be highly readable and motivating. Once the system calculates how far the left side moved (Delta L) and how far the right side moved (Delta R), it compares them directly. The weaker side's movement is divided by the stronger side's movement.

$$\text{Symmetry \%} = [\text{Minimum}(\text{Delta L}, \text{Delta R}) / \text{Maximum}(\text{Delta L}, \text{Delta R})] \times 100$$

Clinical Example: A patient attempts a wide smile. The software tracks the right side of the mouth extending by 30 pixels. The paralyzed left side struggles and only extends by 10 pixels.

The calculation is $(10 / 30) \times 100 = 33.3\%$.

The patient is instantly shown a Symmetry Score of 33%. As neuroplasticity occurs and the nerve heals over the next month, that 10-pixel delta will slowly increase to 15, then 25, driving the Symmetry Score upward, providing undeniable, data-driven proof of recovery.

5.3 The Finite State Machine (FSM) Logic

To ensure the patient performs the exercises in the correct order, the software is governed by a Finite State Machine. The program rests in an 'IDLE' state until the user provides a keyboard interrupt (pressing the 'Enter' key).

Pressing 'Enter' transitions the state from 'IDLE' to 'CALIBRATE', then to 'RECORD_SMILE', then to 'RECORD_BROW', and so on. This prevents the system from accidentally recording a sneeze or a yawn as an exercise, ensuring only intentional physical therapy movements are passed into the mathematical models.

6. USER INTERFACE AND AR INTEGRATION

While a full graphical React web dashboard is planned for Phase 2, the current system requires an interface to communicate with the user. This is achieved through an Augmented Reality (AR) Overlay.

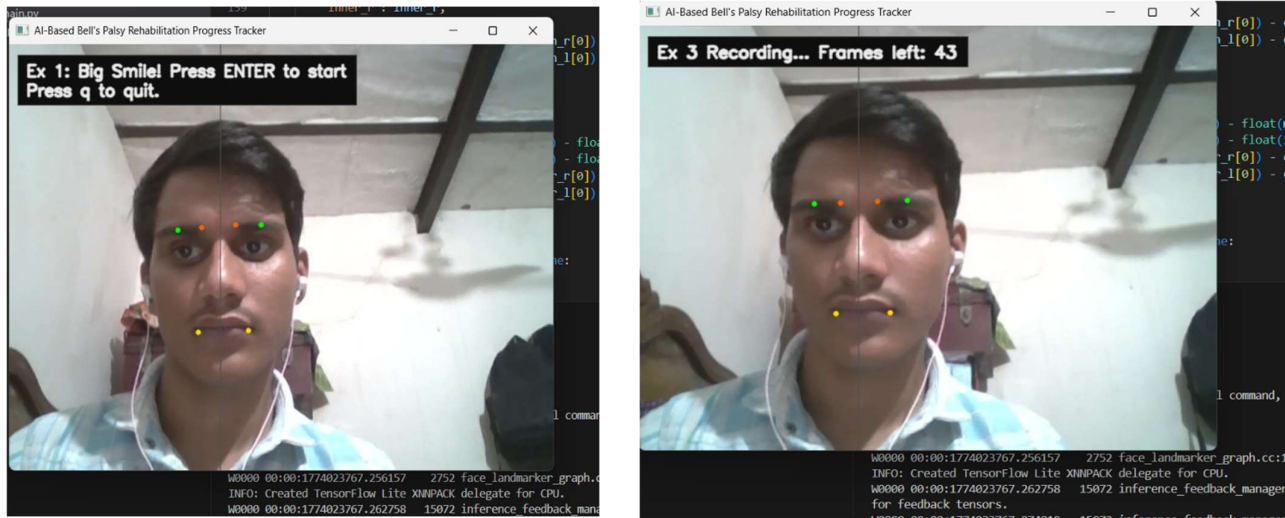
Using OpenCV's internal drawing functions, the Python script renders text and graphics directly onto the live, mirrored video feed.

- **Instructional Prompts:** At the top of the screen, the system prints the current FSM state, instructing the user exactly what to do (e.g., "Press Enter to Lock Baseline", "Exercise 1: Smile as Wide as Possible").
- **Visual Tracking Confirmation:** The user sees neon green tracking circles rendered directly over their facial features in real-time. This provides immense psychological reassurance to the patient, proving that the AI is actively monitoring their specific facial geometry.
- **Diagnostic Output:** Upon completion of an exercise, the calculated Symmetry Percentages are printed centrally on the screen, instantly highlighting the kinematic deficit on the paralyzed side.

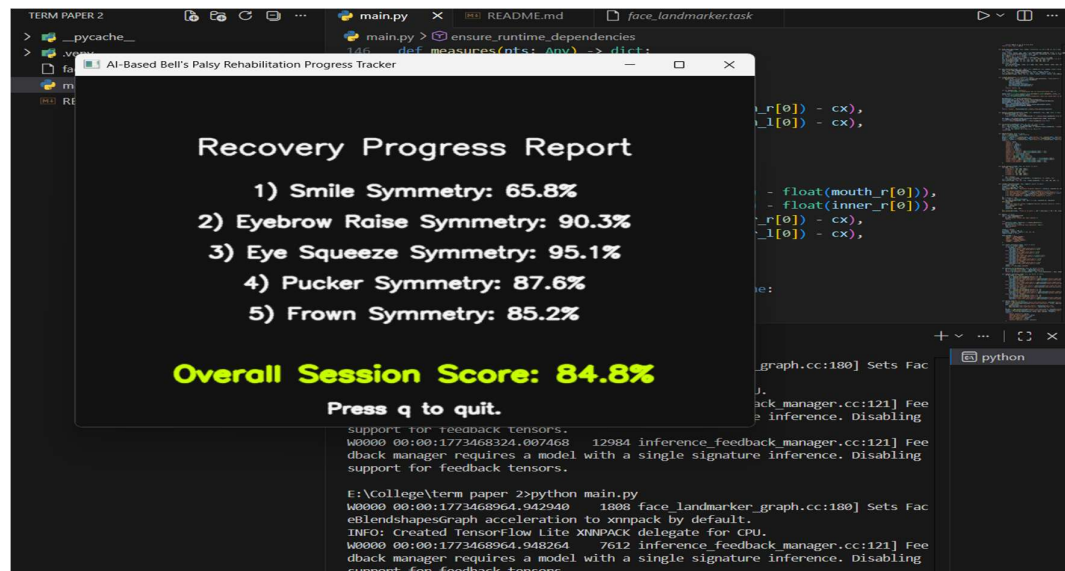
7. Current working

7.1 Phase 1: Video capturing and score generation

The program is successfully , capturing the real time footage ..



Total it uses 5 exercises(5 steps) to do the calculations. For the convenience of User the script made fully controllable and accompanying users pace. To move forward with every step user needs to press enter , then the script records 90 frames for processing and final calculations ..



The Final Scores are shown at the end , exercise wise as well as overall score, so that user can easily understand on which exercise to work more .. And for a normal person it's giving an expected score of 80+..

7.2 Phase 2: Data Storing and Visualization

The latest development phase bridges the backend logic with a frontend interface.

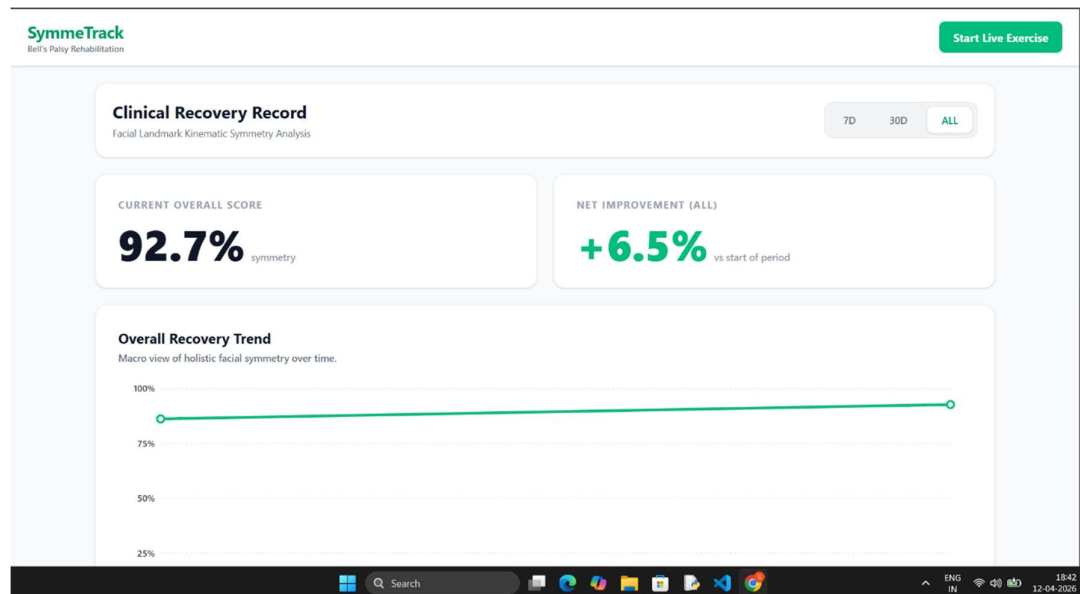
A. JSON Serialization

Upon completing the five core exercises (Smile, Brow Raise, Eye Squeeze, Pucker, and Frown), the Python backend aggregates data. The algorithm identifies the minimum and maximum deltas, dynamically assigns the healthy hemisphere as the denominator, and generates a polarity-agnostic 0-100% symmetry score. The system then serializes these metrics, alongside a timestamp, and appends them to a local `recovery_history.json` ledger.

B. React Clinical Dashboard

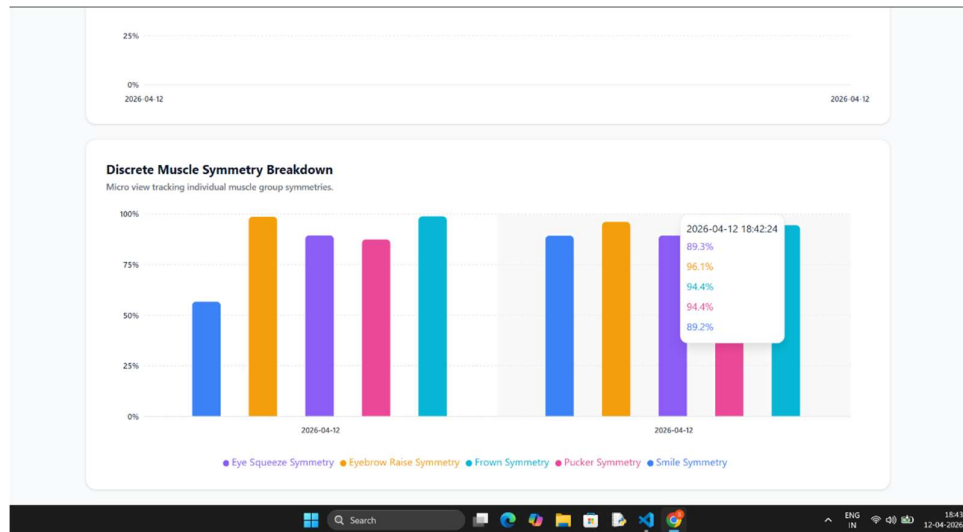
The project have a React.js dashboard to visualize the serialized data. The frontend utilizes the Recharts library to render interactive clinical graphics. The architecture flattens the nested JSON array and slices it based on user-selected timeframes (7-Day, 30-Day, or All-Time).

The dashboard provides two primary analytical views:



1. **Macro Analytics:** A line chart visualizes the "Overall Session Score," allowing patients to track holistic neurological recovery over time.
2. **Micro Diagnostics:** A grouped bar chart breaks down the symmetry of individual muscle groups per session, allowing doctors to identify specific localized weaknesses.

The project has achieved all primary Phase 1 and Phase 2 objectives:



- **Real-Time Tracking:** The Python application successfully tracks five core exercises at 30 FPS on standard consumer CPUs without requiring discrete GPU hardware.
- **Robust Math Models:** The system successfully filters optical noise and corrects for patient posture shifts.
- **Frontend Integration:** The React dashboard successfully mounts, parses the Python-generated JSON data, and renders interactive, clinical-grade UI components.
- **Version Control:** I have initialized and populated the complete source code, neural network models, and README.md documentation in the public GitHub repository (shashankp89/Bell-s-Palsy-Rehabilitation-Assistant).

7.3 Limitations

Due to strict ethical constraints regarding patient data privacy, medical liability, clinical trials on active Bell's Palsy patients have not yet been conducted.

Instead, the vision engine was strictly validated using synthetic testing methodologies. This involves "Self-Simulation," where the developer acts as the patient and intentionally induces asymmetrical facial expressions to verify that the mathematical models react correctly.

Or, making some changes in the script to take pre recorded vedio files of the patients and feeding it to the program to check the expected outcomes..

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

Phase 1 of the AI-Based Bell's Palsy Rehabilitation Assistant has been successfully designed, coded, and validated. The Python-based computer vision engine operates exactly as architected. It successfully interfaces with standard webcam hardware, utilizes advanced CNN

clustering to track 468 points of facial geometry, filters out sensor noise, and applies robust mathematical distance formulas to output a reliable, objective Symmetry Score. The system successfully solves the fundamental problem of subjective "mirror guessing" by providing patients with an objective, data-driven metric for their physical therapy progress.

Phase 2 of the AI-Based Bell's Palsy Rehabilitation Assistant successfully bridges computer vision algorithms with medical physical therapy requirements. By decoupling the AI inference engine from the kinematic math, and bridging the Python backend to a React frontend, the project establishes a highly modular, scalable software architecture. The current prototype proves that edge-computing hardware can provide patients with the accurate, quantitative recovery metrics missing from traditional mirror-based therapy.

8.2 Future Scope: Database integration and Scaling (Phase 3)

With the mathematical processing engine and presentation finalized, the project will transition into the data-handling and production phase:

1. **Cloud Database Integration:** The architecture will replace the local JSON ledger with a secure, real-time NoSQL cloud database (Google Firebase). This will allow doctors to monitor remote patients via web portals asynchronously.
2. **Clinical Report Generation:** I will integrate html2canvas and jsPDF into the React frontend. This feature will allow users to export monthly recovery charts as formatted PDF documents for Electronic Health Record (EHR) attachment.
3. **Video Dataset Testing:** Before attempting trials on live patients, the system will be modified to ingest pre-recorded .mp4 videos of actual Bell's Palsy patients found in open-source medical datasets to fine-tune the tracking sensitivity of the algorithms.
4. **Cross-Platform Mobile Edge Deployment:** To maximize accessibility, the edge-computing inference pipeline will be ported from desktop architectures to mobile environments. Utilizing frameworks such as React Native or Flutter, alongside the MediaPipe iOS/Android APIs, the system will allow patients to perform clinically validated kinematic tracking directly on their smartphones, completely removing the dependency on desktop hardware.

9. REFERENCES

1. Bradski, G., "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
2. Lugaresi, C. et al., "MediaPipe: A Framework for Building Perception Pipelines," *Google Research*, 2019.
3. Soukupova, T. and Cech, J., "Real-Time Eye Blink Detection using Facial Landmarks," *21st Computer Vision Winter Workshop*, 2016.
4. House, J. W., & Brackmann, D. E., "Facial nerve grading system," *Otolaryngology–Head and Neck Surgery*, 1985.
5. Harris, C. R., et al., "Array programming with NumPy," *Nature*, 2020.